

## Módulo 2

### M2.1 Visão geral da plataforma Microsoft .NET

#### O que é .NET?

.NET é uma "plataforma de software". É um ambiente independente de linguagem para criação de programas que possam interoperear de serem desenvolvidos para uma combinação hardware/software particular, os programas serão desenvolvidos para o ".NET", onde o .NET estiver implementado. .NET também é o nome coletivo dado aos vários bits de software construídos sob a plataforma Studio.NET e Windows.NET Server, por exemplo) quanto os serviços (como Passport, HailStorm e outros). Os componentes que são chamados coletivamente de .NET Framework.

#### O .NET Framework

O .NET Framework tem duas partes principais:

- \* A Common Language Runtime (CLR).
- \* Um conjunto hierárquico de biblioteca de classes.

A CLR é descrita como o "motor de execução" do .NET. Ela fornece o ambiente dentro do qual os programas executam. As suas principais funções são:

- \* Conversão de linguagem de baixo do estilo assembly, chamada de Intermediate Language (IL), para código nativo da plataforma.
- \* Gerenciamento de memória, incluindo garbage collection.
- \* Verificação e reforço de restrições de segurança no código em execução.
- \* Carregamento e execução de programas, com controle de versão e outras características.

Antes de entrarmos em maiores detalhes sobre esses assuntos, alguns termos devem ser esclarecidos.

#### Código Gerenciado

"Código Gerenciado" (Managed Code) é o código para o .NET, e que contém certas informações extras - "metadados" - para descrever o código gerenciado quanto o não-gerenciado possam executar em tempo de execução, apenas o código gerenciado contém a informação, por exemplo, uma execução segura de da interoperabilidade.

#### Dados Gerenciados

Com o Código Gerenciado vem os Dados Gerenciados. A CLR fornece facilidades para alocação e desalocação de memória, além de que o .NET usam Dados Gerenciados por default - C#, Visual Basic.NET, JScript.NET — entre outras — C++ — não fornece. Programa em linguagem que você estiver utilizando, impor certas restrições na disponibilidades de alguns recursos; por exemplo, C++ perde he controle do código gerenciado e não-gerenciado, alguém pode ter tanto dados gerenciados quanto não-gerenciados em aplicações.

#### Sistema de Tipos Comum

A CLR usa uma coisa chamada de Common Type System (CTS) para reforçar o "type-safety". Isto garante que todas as classes sejam descrevendo tipos em uma forma comum. CTS define como os tipos funcionam dentro do runtime (sua declaração e uso), quais tipos de uma outra linguagem, incluindo tratamento de exceção através de linguagens. Assim como garante que tipos sejam utilizados, o runtime também garante que o código não tente acessar memória que não foi alocada para ele (isto é dizer que o código é type-safe).

#### Assemblies

Programas .NET são construídos de "Assemblies". Um Assembly é uma coleção de código e metadados compilados e classificados por funcional atômica. Todos os Assemblies contêm um Manifesto, o qual contém o nome do Assembly, versão e localidade, uma lista de quais dependências o Assembly tem, e quais características são exportadas pelo Assembly. Quando você compila algo para o .NET, Em vez disso, é um pequeno wrapper de PE em volta de três blocos de dados. PE (Portable Executable) é o formato binário usado para Arquivos PE contêm um programa MS-DOS stub (se você já tentou executar um executável Win32 do DOS e viu a mensagem "This Windows", (você já viu um stub em ação). Binários do .NET então conterão um stub de Win32 - para ou utilizar o .NET para executar como,

"This program requires .NET to run" se ele não estiver disponível.

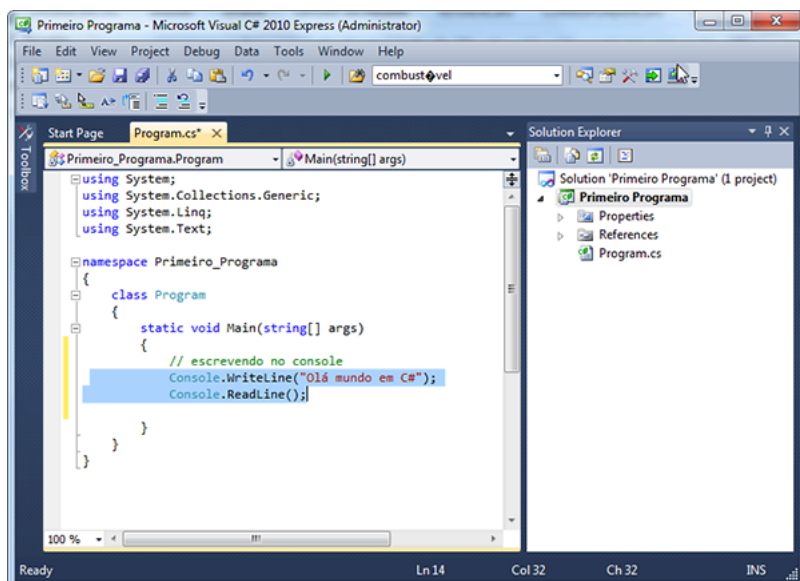
O primeiro bloco de dados dentro do wrapper PE é a própria IL. IL se parece muito com o assembly. Isto é o que realmente é compilado chamado de metadado. Ele descreve o conteúdo do arquivo - por exemplo, quais métodos ele disponibiliza, quais parâmetros eles recebem, o terceiro é o manifesto. Ele descreve de quais outros componentes o executável precisa para executar. Ele também contém chaves de forma que a CLR possa garantir que o componente externo é aquele que o executável pensa que ele é. Ao executar o executável, o Time. A medida que cada método dentro do executável é chamado, ele é compilado para o código nativo; chamadas subsequentes não têm o mesmo peso da mesma compilação, então o overhead é pago apenas uma vez. Compilação JIT levanta algumas questões. Uma é que ela requer processador especializado. Para resolver isto, a MS disponibilizou dois compiladores JIT: um normal, que otimiza o código compilado caro em termos de processador e memória; e um "EconoJIT". EconoJIT pode não otimizar o código tão bem, mas irá requerer menos processamento para executar. Também permitirá que a execução do seu programa descarte a forma compilada de um método, liberando de compilá-la novamente. Um terceiro tipo de compilador sugerido (embora não tenha sido incluído na versão 1), é chamado "OptJIT" diferente: usaria um subconjunto da IL (OptIL) com informação adicional para sugerir para o compilador OptJIT como gerar sua melhor otimização da IL. A idéia é reduzir o overhead devido à compilação JIT sem sacrificar a qualidade do código de máquina emitido.

### M2.2 Implementação de técnicas de programação orientada a objetos em C#

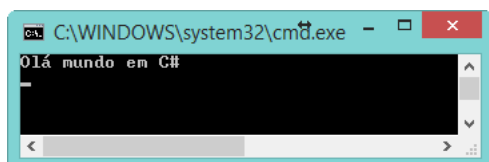
A linguagem C# (pronuncia-se "C Sharp") faz parte desse conjunto de ferramentas oferecidas na plataforma .NET e surge como um exemplo orientada a objetos, fortemente tipada e altamente escalável a fim de permitir que uma mesma aplicação possa ser executada em independentemente deles serem PCs, *handhelds* ou qualquer outro dispositivo móvel. Além do mais, de acordo com Lima e Reis (2002) como objetivo permitir o desenvolvimento de qualquer tipo de aplicação: *web services*, Windows convencional, aplicações para ser *handheld* e também para internet, entre outros.

A proposta do C# em termos de linguagens de programação poderia ser descrita esboçando algumas das suas características principais.

- **Clareza, simplicidade e facilidade:** C# é clara, simples, fácil de aprender, mas nem por isso menos poderosa.
- **É completamente orientada a objetos:** C#, diferentemente de muitas linguagens existentes no mercado, é completa e tudo é um objeto.
- **Não requer ponteiros para gerenciar a memória:** C# não requer ponteiros para alocar/desalocar memória *heap*. Isso, acima, é feito pelo GC (*Garbage Collector*).
- **Suporta interfaces, sobrecarga, herança, polimorfismo, atributos, propriedades e coleções, dentre outras características da linguagem que se diz orientada a objetos.**
- **Possui código 100% reutilizável:** todo programa desenvolvido em C# é passível de reutilização a partir de qualquer ponto (LIMA; REIS, 2002).
- A seguir, temos um trecho simples de código em C#:



O programa da figura anterior escreve a frase "Olá mundo em C#" em uma janela de console (*prompt* de comando). Ao executarmos o programa, vamos estudá-la.



Tela de saída do primeiro programa

Em sua primeira linha, temos a cláusula (ou palavra reservada ou comando) *using*. Sua função aqui é carregar os *namespaces* (classe) para a execução do programa.

Após a definição das bibliotecas que serão utilizadas, iniciaremos a estrutura do programa. O programa principal também é considerado uma classe, utilizamos a cláusula *class* seguida pelo nome da classe, que, neste caso, será o nome do programa.

As chaves servem para definir o escopo de cada bloco de codificação, ou seja, determinam o início e o fim de cada sequência de código.

Por se tratar do programa principal, essa classe conterá apenas um método: *Main()*. É importante entender que a estrutura do programa principal é assim, com ao deste exemplo!

A cláusula *static* existe ali para indicar ao .NET que este é o método principal e que esse é o ponto inicial do programa. Já a cláusula *void* possui nenhum retorno.

O método *Main()* escrito conforme o exemplo significa também que o programa não receberá nenhum argumento (ou parâmetros). Se necessário que o programa receba esse tipo de informação, a declaração desse método sofre uma ligeira alteração:

```
static void Main(string[] args)
```

Por fim, devemos nos atentar ao ponto-e-vírgula (;). Esse caractere deve ser colocado no final de cada linha de comando, indicando o fim da linha.

Se tratando de organização de código, temos os comentários, que podem ser escritos em linhas ou em blocos. Para comentários em linha, usamos a sequência */\* \*/*, conforme o exemplo:

```
static void Main(string[] args)
{
    /*
    escrevendo no console
    Exemplo de declaração de método Main com
    Parâmetro: Main(string[] args)
    Não se esqueça de colocar um ; no final de cada
    linha de comando!
    */
    Console.WriteLine("Olá mundo em C#");
    Console.ReadLine();
}
```

### Primeiro programa

Os comentários são observações que servirão unicamente para que o desenvolvedor faça anotações sobre seu trabalho ou sobre o do comentário (seja em bloco ou em linha) será ignorado pelo compilador (incluindo comandos).

Para fins didáticos, utilizaremos o console para que possamos exemplificar os códigos e testarmos os conceitos. Isso torna o apren realmente é necessário.

Vejamos agora outro exemplo de código:

```
static void Main(string[] args)
{
    char c;
    string str;
    // Escreve no console sem retorno de carro
    Console.Write("Digite seu nome: ");
    // Lê uma string do console.<Enter> para concluir
    str = Console.ReadLine();
    // Escreve no console sem retorno de carro
    Console.Write("Digite uma letra:");
    // Lê do console um caractere simples.
    c = (char)Console.Read();
    // Escreve uma linha em branco
    Console.WriteLine();
    // Escreve uma string no console
    Console.WriteLine("Seu nome é: {0}", str);
    // Escreve 1 caractere com ToString() para converter
    Console.WriteLine("A letra é: {0}", c.ToString());
    Console.ReadLine();
}
```

### Código com os comentários

Neste exemplo, encontramos uma rica quantidade de conceitos utilizados para programar em C#. Diferente do exemplo anterior, e solicita a entrada de informações e devolve essas informações na tela.

Para isso, utilizamos a classe Console, que possui os métodos necessários para exibição de informações na tela e captura de dados: quadro a seguir:

### Quadro 3 – Métodos de interação de teclado e tela

| Método                   | Descrição                                                                                  |
|--------------------------|--------------------------------------------------------------------------------------------|
| <b>Console.Write</b>     | Escreve na tela, na posição onde está o cursor.                                            |
| <b>Console.WriteLine</b> | Escreve na tela e ao final salta uma linha, posicionando o cursor no início da nova linha. |
| <b>Console.Read</b>      | Lê a digitação de algo no teclado (aguarda até que uma tecla seja pressionada).            |
| <b>Console.ReadLine</b>  | Lê a digitação do teclado e aguarda até que a tecla Enter seja digitada.                   |

Outro conceito mostrado no exemplo MeuTeste2 é o das variáveis, que terão o escopo de onde elas foram definidas. No exemplo I declaradas (c e str) funcionam (escopo) dentro do método Main.

O quadro a seguir mostra os tipos de variáveis utilizados em C#.

### Quadro 4 – Tipos de variáveis em C#

| Tipo            | Valore possíveis                                                 |
|-----------------|------------------------------------------------------------------|
| <i>bool</i>     | Verdadeiro ou falso (valores booleanos)                          |
| <i>byte</i>     | 0 a 255 (8 bits)                                                 |
| <i>sbyte</i>    | -128 a 127 (8 bits)                                              |
| <i>char</i>     | Um caractere (16 bits)                                           |
| <i>decimal</i>  | $\pm 1.0 \times 10^{-28}$ a $\pm 7.9 \times 10^{28}$ (128 bits)  |
| <i>double</i>   | $\pm 5.0 \times 10^{-324}$ a $\pm 1.7 \times 10^{308}$ (64 bits) |
| <i>float</i>    | $\pm 1.5 \times 10^{-45}$ a $\pm 3.4 \times 10^{38}$ (32 bits)   |
| <i>int</i>      | -2.147.483.648 a 2.147.483.647 (32 bits)                         |
| <i>uint</i>     | 0 a 4.294.967.295 (32 bits)                                      |
| <i>Long[L4]</i> | 9.223.373.036.854.775.808 a 9.223.372.036.854.775.807 (64 bits)  |
| <i>ulong</i>    | 0 a 18.446.744.073.709.551.615 (64 bits)                         |
| <i>object</i>   | Qualquer tipo                                                    |
| <i>short</i>    | -32.768 a 32.767 (16 bits)                                       |
| <i>ushort</i>   | 0 a 65.535 (16 bits)                                             |
| <i>string</i>   | Sequência de caracteres (16 bits por caractere)                  |

Para se trabalhar com variáveis, devemos tomar alguns cuidados e seguir algumas regras:

- O primeiro caractere deve ser uma **letra**.
- Se houver mais de um caractere, só podemos usar **letra** ou **algarismo**.
- Nomes de variáveis escritas com letras maiúsculas serão diferentes de letras minúsculas. Lembre-se: media é **diferente** de média.
- Nenhuma palavra reservada (nomes de comandos ou cláusulas) poderá ser nome de variável.

Exemplos:

```
int x;  
bool opcao;  
string texto1;
```

Apenas após a definição da variável é que podemos utilizá-las em nosso programa. Para atribuímos valores para as nossas variáveis usamos a igualdade (=).

Exemplos:

```
x = 10; //Lê se: a variável x recebe o valor 10, ou, x recebe 10  
opcao = false;  
texto1 = "Boa noite";
```

Agora que a estrutura básica de um programa em C# foi apresentada, vejamos o próximo código. Este exemplo mostra a idade at no ano que vem:

```
static void Main(string[] args)
{
    int idade;
    idade = 15;

    // escrevendo no console
    Console.WriteLine("Programa Idade versão 1.0");
    Console.WriteLine();
    Console.WriteLine("A idade atual é: {0}", idade);
    Console.WriteLine();
    idade = idade + 1;
    Console.WriteLine("A idade ano que vem é: {0}", idade);
}
```

Idade\_v1

O programa Idade\_v1, embora simples, ilustra mais alguns importantes fundamentos. Esse programa é iniciado declarando uma variável (`int idade;`) e logo em seguida essa variável é iniciada (ou populada, de popular) com o valor 15 (`idade = 15;`), presumindo que a

Como o objetivo do programa é calcular a idade da pessoa para o ano que vem, é realizada uma operação matemática de adição (adicionado um ao valor já existente na variável `idade`, ou seja, a idade para o ano que vem é 16.

Em C#, é possível realizar operações matemáticas, conforme o quadro a seguir.

Quadro 5 – Operadores matemáticos em C#

#### SímboloOperador matemático

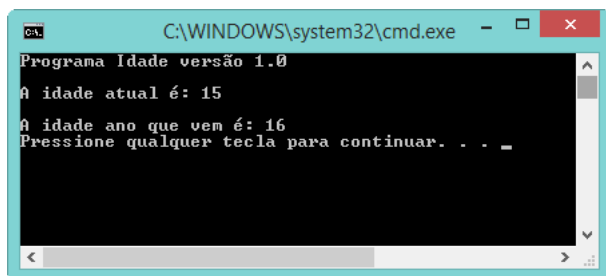
|   |                  |
|---|------------------|
| + | Adição           |
| - | Subtração        |
| * | Multiplicação    |
| / | Divisão          |
| % | Resto da divisão |

Os programas MeuTeste2 e Idade\_v1 também introduzem o conceito de máscara de exibição de valores (`{0}`). Essa máscara permite a saída, deixando as mensagens mais simpáticas ao usuário do programa.

O conceito do uso desta máscara é bem simples. O conteúdo das chaves (`{0}`) é substituído pelo conteúdo da variável passada para a

```
Console.WriteLine("A idade ano que vem é: {0}", idade);
```

Ela vai ser exibida para o usuário da seguinte maneira:



Saída do programa V1.00

Podem ser montadas várias máscaras em uma mesma mensagem de exibição, conforme podemos ver nessa versão melhorada do

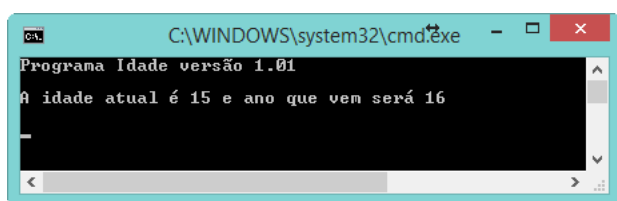
```
static void Main(string[] args)
{
    // declara a idade
    int idade, idadeanoquevem;
    idade = 15;
    idadeanoquevem = idade + 1;
    // escrevendo no console
    Console.WriteLine("Programa Idade versão 1.01");
    Console.WriteLine();
    Console.WriteLine("A idade atual é {0} e ano que vem será {1}",
        idade, idadeanoquevem);
    Console.WriteLine();
}
```

idade\_V1.01

Para a versão nova, o programa Idade\_v101 teve duas alterações. Para ilustrar o exemplo da máscara na mensagem de exibição e a necessidade de criar uma segunda variável para armazenar a idade para o ano que vem. Deste modo, o cálculo da idade passou a *idadeanoquevem* e a idade atual da variável *idade*[L8] . Essa divisão do cálculo em duas variáveis traz uma maior organização ao código e uma mensagem única de saída com as suas respostas. Assim, a mensagem é definida como:

```
Console.WriteLine("A idade atual é {0} e ano que vem será {1}",
    idade, idadeanoquevem);
```

Ela será exibida ao usuário da seguinte forma:



Saída do programa Versão 1.01

Para a leitura de um valor via teclado, emprega-se a instrução `Console.ReadLine();`. Uma forma direta de aplicação é a atribuição de uma variável que receberá esse conteúdo.

```
nome = Console.ReadLine();
```

Dessa forma, o valor digitado pelo usuário será atribuído à variável "nome". No entanto, tal forma de aplicação é viável e fácil de ser utilizada, pois a variável é um *string*, pois sua aplicação sempre retornará um valor desse tipo. Para os conteúdos de variáveis de outro tipo, faz-se necessário converter o valor lido para o tipo desejado, antes de sua atribuição.

```
float x = Convert.ToSingle(Console.ReadLine());
```

Observe que é feita a leitura do conteúdo da variável por meio do argumento da instrução para conversão do tipo *String* para *float*.

A forma de aplicação dessa instrução, sob o aspecto prático, será mais bem ilustrada na próxima atividade em laboratório[L9] . O conteúdo abordará as formas de conversão de *String* para outro tipo de variável.

Quadro 6 – Conversões de *String* para tipos numéricos.

| Tipo numérico  | Método                                 | Método Parse                |
|----------------|----------------------------------------|-----------------------------|
| <i>decimal</i> | <code>Convert.ToDecimal(String)</code> | <code>Decimal.Parse</code>  |
| <i>float</i>   | <code>Convert.ToSingle(String)</code>  | <code>Single.Parse()</code> |
| <i>double</i>  | <code>Convert.ToDouble(String)</code>  | <code>Double.Parse()</code> |
| <i>short</i>   | <code>Convert.ToInt16(String)</code>   | <code>Int16.Parse()</code>  |
| <i>int</i>     | <code>Convert.ToInt32(String)</code>   | <code>Int32.Parse()</code>  |
| <i>long</i>    | <code>Convert.ToInt64(String)</code>   | <code>Int64.Parse()</code>  |
| <i>ushort</i>  | <code>Convert.ToUInt16(String)</code>  | <code>UInt16.Parse()</code> |
| <i>uint</i>    | <code>Convert.ToUInt32(String)</code>  | <code>UInt32.Parse()</code> |

*ulong*      `Convert.ToInt64(String) UInt64.Parse()`

Existe outra importante maneira de fazer conversões de tipo, o modelador *cast*.

O *cast* é um método pelo qual um valor é convertido a partir de um tipo de dados para outro. Ele pode ser executado para conversão de destino é de menor precisão ou que tenha um intervalo de valores menor. Também é utilizado para a conversão de instância de classe.

Muitas vezes o *cast*, por ser uma conversão explícita onde o compilador é informado sobre a mudança, pode resultar em erros de tipos. Para testar a compatibilidade antes de realmente executar um *cast*, o C# fornece dois operadores:

- O operador *'is'* verifica a possibilidade de fazer o *cast*, além de ser utilizado para determinar um tipo de objeto é do tipo de destino.
- O operador *'as'* é utilizado para se obter o valor de *cast* sem efetivar a mudança de tipo. Dessa forma, é possível determinar com sucesso.

Voltando ao programa da idade, vamos agora realizar mais algumas implementações, de modo que ele fique mais prático, pois, em alguns conceitos importantes, o programa não possui uma utilidade muito prática. Vejamos agora o programa *Idade\_v2* (versão 2.0).

```
static void Main(string[] args)
{
    // declara a idade
    int idade, idadenofuturo, qtdeanos;
    idade = 0;
    qtdeanos = 0;
    // escrevendo no console
    Console.WriteLine("Programa Idade versão 2.0");
    Console.WriteLine();
    Console.Write("Informe sua idade atual: ");
    idade = Convert.ToInt32(Console.ReadLine());
    Console.WriteLine();
    Console.Write("Daqui a quantos anos você quer saber qual será sua idade? ");
    qtdeanos = Convert.ToInt32(Console.ReadLine());
    idadenofuturo = idade + qtdeanos;
    Console.WriteLine("Hoje você tem {0} anos. Daqui a {1} anos você terá {2} anos de idade!",
        idade, qtdeanos, idadenofuturo);
}
```

Observe que é feita a leitura do conteúdo da variável por meio do argumento da instrução para conversão do tipo *String* para *float*.

A forma de aplicação dessa instrução, sob o aspecto prático, será mais bem ilustrada na próxima atividade em laboratório[L9]. O exemplo mostra as formas de conversão de *String* para outro tipo de variável.

Quadro 6 – Conversões de *String* para tipos numéricos.

| Tipo numérico  | Método                                 | Método Parse                 |
|----------------|----------------------------------------|------------------------------|
| <i>decimal</i> | <code>Convert.ToDecimal(String)</code> | <code>Decimal.Parse()</code> |
| <i>float</i>   | <code>Convert.ToSingle(String)</code>  | <code>Single.Parse()</code>  |
| <i>double</i>  | <code>Convert.ToDouble(String)</code>  | <code>Double.Parse()</code>  |
| <i>short</i>   | <code>Convert.ToInt16(String)</code>   | <code>Int16.Parse()</code>   |
| <i>int</i>     | <code>Convert.ToInt32(String)</code>   | <code>Int32.Parse()</code>   |
| <i>long</i>    | <code>Convert.ToInt64(String)</code>   | <code>Int64.Parse()</code>   |
| <i>ushort</i>  | <code>Convert.ToUInt16(String)</code>  | <code>UInt16.Parse()</code>  |
| <i>uint</i>    | <code>Convert.ToUInt32(String)</code>  | <code>UInt32.Parse()</code>  |
| <i>ulong</i>   | <code>Convert.ToUInt64(String)</code>  | <code>UInt64.Parse()</code>  |

Existe outra importante maneira de fazer conversões de tipo, o modelador *cast*.

O *cast* é um método pelo qual um valor é convertido a partir de um tipo de dados para outro. Ele pode ser executado para conversão de destino é de menor precisão ou que tenha um intervalo de valores menor. Também é utilizado para a conversão de instância de classe.

Muitas vezes o *cast*, por ser uma conversão explícita onde o compilador é informado sobre a mudança, pode resultar em erros de tipos. Para testar a compatibilidade antes de realmente executar um *cast*, o C# fornece dois operadores:

- O operador `'is'` verifica a possibilidade de fazer o *cast*, além de ser utilizado para determinar um tipo de objeto é do tipo de destino.
- O operador `'as'` é utilizado para se obter o valor de *cast* sem efetivar a mudança de tipo. Dessa forma, é possível determinar com sucesso.

Voltando ao programa da idade, vamos agora realizar mais algumas implementações, de modo que ele fique mais prático, pois, em alguns conceitos importantes, o programa não possui uma utilidade muito prática. Vejamos agora o programa Idade\_v2 (versão 2.0).

```

C:\WINDOWS\system32\cmd.exe
Programa Idade versão 2.0
Informe sua idade atual: 23
Daqui a quantos anos você quer saber qual será sua idade? 13
Hoje você tem 23 anos. Daqui a 13 anos você terá 36 anos de idade!
Pressione qualquer tecla para continuar. . . _
  
```

Saída da versão 2.0

Estudando o código do programa Idade\_v2, devemos nos atentar às variáveis. Embora seus valores sejam informados pelo usuário logo após sua declaração.

Faz parte das boas práticas de programação iniciar as variáveis com valores predeterminados para garantir os valores iniciais das variáveis. Quando declaramos uma variável e esta não é inicializada pelo programador, as variáveis primitivas recebem um valor *default*. Outras variáveis declaradas ou instanciadas, recebem como valor *default* o *null*. Veja:

Quadro 7 – Padrões de valores de tipos de variáveis em C#

#### Tipo      Conteúdo

*Boolean* **False**

*Byte*    **0**

*Short*   **0**

*Char*    *Null*

*Int*      **0**

*Float*   **0.0**

*Long*    **0**

*Double* **0.0**

Agora sabemos como é a estrutura básica de um programa em C#, como se comportam as variáveis e como fazer o programa interagir. O exemplo ilustrará como trabalhar com textos (*string*):

```
static void Main(string[] args)
{
    string c, c1, d, d1;
    Console.WriteLine("\n Digite a palavra 1: ");
    c1 = Console.ReadLine();
    Console.WriteLine("\n Digite a palavra 2: ");
    c1 = Console.ReadLine();
    Console.WriteLine("\n tamanho da 1 palavra: {0}", c.Length);
    Console.WriteLine("\n concatenando sem armazenar: {0}", c + c1);
    d = c;
    Console.WriteLine("\n o conteúdo de d: {0}", d);
    d1 = c + c1;
    Console.WriteLine("\n concatenação: {0}", d1);
    Console.WriteLine("\n primeiro caractere: {0}", c.Substring(0, 1));
    Console.WriteLine("\n último caractere: {0}", c1.Substring(c1.Length - 1, 1));
    Console.WriteLine("\n todos menos o primeiro: {0}", d1.Substring(1));
    Console.WriteLine("\n o terceiro elemento: {0}", c.Substring(4, 1)); // 5º caracter
    Console.WriteLine("\n os três primeiros elementos: {0}", c.Substring(0, 3));
    Console.WriteLine("\n os três últimos elementos: {0}", c1.Substring(c1.Length - 3, 3));
}
}
```

## Programatexto

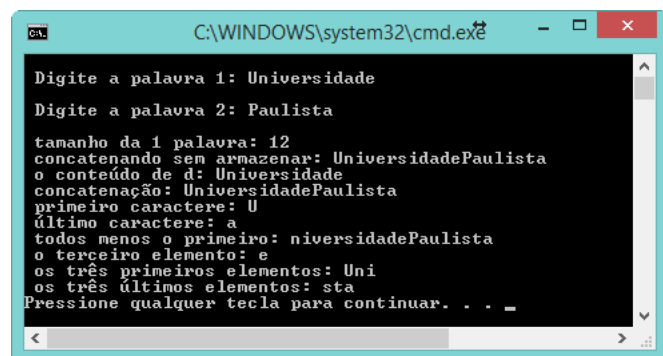
O programa Programatexto ilustra alguns exemplos de como trabalhar com dados do tipo texto (*string*), verificando o tamanho do diferentes e trabalhando com partes dos textos, conforme mostra o quadro a seguir.

### Quadro 8 – Métodos de manipulação de dados do tipo texto (*string*)

| Função                                     | Descrição                                                                                                                                                                                                                                                                                     |
|--------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Substring(posição_inicial, tamanho)</b> | Este método retorna um pedaço do texto ( <i>substring</i> ) contido na variável (ou atributo) do tipo <i>string</i> posição de caractere especificada (posição_inicial) e tem um comprimento especificado (tamanho) da <i>string</i> (primeiro caractere do texto) é indicada como posição 0. |
| <b>Length</b>                              | Esta propriedade (atributo) contém o número de caracteres – ou seja, o tamanho – do texto ( <i>string</i> )                                                                                                                                                                                   |

Algumas observações importantes devem ser feitas em relação ao tipo de dado *string*. Em C#, *strings* são objetos do tipo *System.String*. A classe *System.String* define métodos para manipular *strings* como concatenação, tamanho da *string*, cópia etc. Em C#, *strings* não são criadas e inicializadas, qualquer modificação efetuada não é feita diretamente na *string*. Entretanto, o método usado na operação não modifica a *string* original. Isso equivale a dizer que operações com *string* são custosas porque além de estarmos lidando com uma operação que modifique a *string* vai retornar um novo objeto (Lima e Reis, 2002).

A saída para as palavras Universidade Paulista inseridos no programa é apresentada na Figura 23



## Saída do Programa Texto

A partir de agora, veremos os aspectos mais “inteligentes” da programação, ou seja, falaremos sobre desvio condicional e laços.

O Desvio condicional ou tomada de decisão, como em qualquer outra linguagem de programação, é a característica responsável por fazer o programa tomar decisões. Vejamos o exemplo a seguir:

```
static void Main(string[] args)
{
    double numero = 0;
    Console.WriteLine("Digite um número: ");
    numero = double.Parse(Console.ReadLine());
    if (numero > 20)
    {
        Console.WriteLine("Metade: {0}", numero / 2);
    }
}
```

## Código 7-Exemplo desvio condicional

O exemplo Testedecondicao ilustra uma maneira muito simples de utilização de um desvio condicional. Após a digitação de um número, o programa verifica se o número digitado é maior que 20 (if (numero > 20)). Sendo verdadeira esta condição, o programa executa o número digitado. Se o número for igual ou menor a 20, o programa não executa nenhum procedimento.

Portanto, um desvio condicional em C# possui a seguinte estrutura:

```
if (condição)
{
    < seqüência de comandos separados por ; >
}
else
{
    < seqüência de comandos separados por ; >
}
```

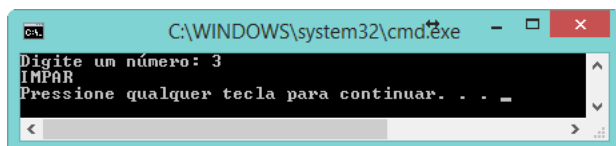
Vejamos agora um outro exemplo de desvio condicional:

```
static void Main(string[] args)
{
    double numero = 0;
    Console.Write("Digite um número: ");
    numero = Convert.ToDouble(Console.ReadLine());
    if (numero % 2 == 0)
    {
        Console.WriteLine("PAR");
    }
    else
    {
        Console.WriteLine("IMPAR");
    }
}
```

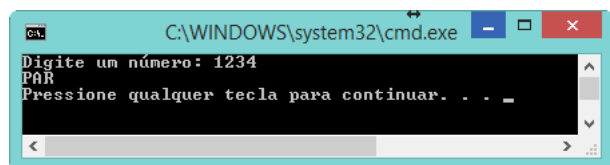
## Testedecondicao2

No exemplo Testedecondicao2, o usuário digita um número e o programa determina, através de um desvio condicional, se o número é par ou ímpar. Se o resto da divisão do número informado for igual a zero faça <condição 1>, se não, faça a <condição 2> é o primeiro bloco de comandos (condição verdadeira) e a <condição 2> refere-se ao segundo bloco de comandos (condição falsa).

Assim ao executarmos o programa, conforme o valor de entrada, mostra um dos valores



saída quando um número ímpar é digitado.



Saída do programa ao digitarmos um número par

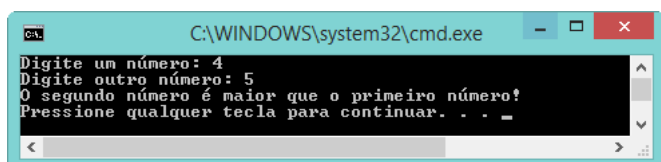
O desvio condicional pode assumir ainda uma terceira estrutura:

```
static void Main(string[] args)
{
    int a, b = 0;
    Console.Write("Digite um número: ");
    a = Convert.ToInt32(Console.ReadLine());
    Console.Write("Digite outro número: ");
    b = Convert.ToInt32(Console.ReadLine());
    if (a > b)
    {
        // Bloco de comandos 1
        Console.WriteLine("O primeiro número é maior que o segundo número!");
    }
    else if (a < b)
    {
        // Bloco de comandos 2
        Console.WriteLine("O segundo número é maior que o primeiro número!");
    }
    else
    {
        // Bloco de comandos 3
        Console.WriteLine("Os dois números são iguais!");
    }
}
```

### Testedeconcao3

O exemplo Testedecondicao3, mostra a estrutura de desvio condicional com três blocos de comandos, sendo:

- O bloco de comandos 1 refere-se a condição verdadeira do primeiro desvio condicional ( $a > b$ ).
- O bloco de comandos 2 refere-se a condição falsa do primeiro desvio condicional ( $a > b$ ) e a condição verdadeira do segundo desvio condicional ( $a < b$ ).
- O bloco de comandos 3 refere-se a condição falsa do primeiro desvio condicional ( $a > b$ ) e a condição falsa do segundo desvio condicional ( $a < b$ ).



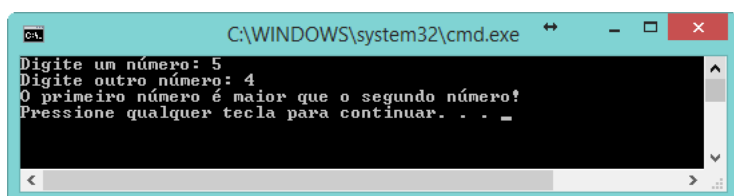
Porém, se analisarmos o mesmo programa com outra estrutura de desvio condicional, tal como:

```
static void Main(string[] args)
{
    int a, b = 0;
    Console.Write("Digite um número: ");
    a = Convert.ToInt32(Console.ReadLine());
    Console.Write("Digite outro número: ");
    b = Convert.ToInt32(Console.ReadLine());
    if (a > b)
    {
        // Bloco de comandos 1
        Console.WriteLine("O primeiro número é maior que o segundo número!");
    }
    if (a < b)
    {
        // Bloco de comandos 2
        Console.WriteLine("O segundo número é maior que o primeiro número!");
    }
    if (a == b)
    {
        // Bloco de comandos 3
        Console.WriteLine("Os dois números são iguais!");
    }
}
```

### testedecondicao4

O programa Testedecondicao4, em termos de objetivo e tratamento lógico, a mesma coisa que o programa Testedecondicao3. Porém, a estrutura de desvio condicional é diferente! Este exemplo é apenas para ilustrar como é possível construir diversas estruturas lógicas para o desvio condicional.

Ao testar o programa Testedecondição4, temos a saída na Figura 26



Saída do programa Testedecondição4 usando 5 e 4 como entrada

Quando falamos em desvio condicional, torna-se necessário falarmos também sobre como operadores lógicos e relacionais, conforme Não é demais lembrar também o comportamento dos operadores lógicos E, OU e NOT, conforme ilustra as Tabela 10, Tabela 11, respectivamente.

Tabela 9-Operadores lógicos e relacionais em C#.

| Operador                       | Tipo       | Descrição                                                                                |
|--------------------------------|------------|------------------------------------------------------------------------------------------|
| ==                             | Relacional | Igual a. Comparação de igualdade.                                                        |
| !=                             | Relacional | Diferente a. Comparação de desigualdade.                                                 |
| >                              | Relacional | Maior que.                                                                               |
| <                              | Relacional | Menor que.                                                                               |
| >=                             | Relacional | Maior ou igual a.                                                                        |
| <=                             | Relacional | Menor ou igual a.                                                                        |
| Conjunção. Operador lógico e.  |            |                                                                                          |
| &&                             | Lógico     | É verdade se todas as Condições forem verdadeiras.                                       |
| Disjunção. Operador lógico ou. |            |                                                                                          |
|                                | Lógico     | É Verdadeira se pelo menos Uma condição for verdadeira.                                  |
| Negação. Operador lógico not.  |            |                                                                                          |
| !                              | Lógico     | Inverte o valor da expressão ou condição, se verdadeira inverte para falsa e vice-versa. |

Tabela 10-Tabela verdade para o operador lógico E (conjunção).

| Condição 1 | Condição 2 | Resultado lógico |
|------------|------------|------------------|
| FALSO      | FALSO      | FALSO            |
| FALSO      | VERDADEIRO | FALSO            |
| VERDADEIRO | FALSO      | FALSO            |
| VERDADEIRO | VERDADEIRO | VERDADEIRO       |

Tabela 11-Tabela verdade para o operador lógico OU (disjunção).

| Condição 1 | Condição 2 | Resultado lógico |
|------------|------------|------------------|
| FALSO      | FALSO      | FALSO            |
| FALSO      | VERDADEIRO | VERDADEIRO       |
| VERDADEIRO | FALSO      | VERDADEIRO       |

VERDADEIROVERDADEIROVERDADEIRO

Tabela 12-Tabela verdade para o operador lógico NOT (negação) aplicado a condições de disjunção (OU).

Condição 1    Condição 2    Resultado lógico negado

FALSO          FALSO          VERDADEIRO

FALSO          VERDADEIROFALSO

VERDADEIROFALSO          FALSO

VERDADEIROVERDADEIROFALSO

Tabela 13-Tabela verdade para o operador lógico NOT (negação) aplicado a condições de conjunção (E).

Condição 1    Condição 2    Resultado lógico negado

FALSO          FALSO          VERDADEIRO

FALSO          VERDADEIROVERDADEIRO

VERDADEIROFALSO          VERDADEIRO

VERDADEIROVERDADEIROFALSO

Existe outro tipo de desvio condicional em C# chamado *Switch* (ou *case*). Este desvio condicional não se utiliza de expressões rela a condição possui um determinado valor para tomar a decisão. Necessariamente, este valor deverá ser um valor do tipo numérico

A estrutura do desvio condicional do tipo *switch* segue o seguinte padrão:

```
switch (condição)
{
    case valor1:
        < sequência de comandos separados por ; >
        break;
    case valor2:
        < sequência de comandos separados por ; >
        break;
    case valorN:
        < sequência de comandos separados por ; >
        break;
}
```

Dentro do bloco principal do *switch* pode haver quantos blocos *case* forem necessários para realizar as devidas comparações. Poré observados:

- Em C#, o comando *break*, no final de cada sequência de comandos, de cada bloco *case*, deve ser inserido, indicando fi respectivo bloco *case*.
- Pode haver um bloco *default* após todos os blocos *case*. Esse bloco será executado caso nenhuma condição *case* seja v
- Não pode haver mais de um bloco *case* verificando o mesmo valor da condição.

```

static void Main(string[] args)
{
    {
        int a, b, c = 0;
        Console.WriteLine("Digite um número: ");
        a = Convert.ToInt32(Console.ReadLine());
        Console.WriteLine("Digite outro número: ");
        b = Convert.ToInt32(Console.ReadLine());
        Console.WriteLine("Agora, o que você quer fazer com esse números? ");
        Console.WriteLine("1 - Verificar se o primeiro número é maior que o segundo número.");
        Console.WriteLine("2 - Verificar se o segundo número é maior que o primeiro número.");
        Console.WriteLine("3 - Verificar se os números são iguais.");
        c = Int32.Parse(Console.ReadLine());
        switch (c)
        {
            case 1:
                if (a > b)
                {
                    Console.WriteLine("O primeiro número é maior que o segundo número!");
                }
                else
                {
                    Console.WriteLine("O primeiro número não é maior que o segundo número!");
                }
                break;
            case 2:
                if (a < b)
                {
                    Console.WriteLine("O segundo número é maior que o primeiro número!");
                }
                else
                {
                    Console.WriteLine("O segundo número não é maior que o primeiro número!");
                }
                break;
            case 3:
                if (a == b)
                {
                    Console.WriteLine("Os dois números são iguais!");
                }
                else
                {
                    Console.WriteLine("Os dois números não são iguais!");
                }
                break;
            default:
                Console.WriteLine("Opção inválida!");
                break;
        }
        Console.WriteLine("Pressione qualquer tecla para sair...");
    }
}

```

### Testeswitch

O programa Testeswitch mostra não só a aplicação do desvio condicional switch, como também o uso integrado de outros desvios case.

É possível verificar que o programa Testeswitch, em termos de escopo, faz a mesma coisa que o programa Testecondicao4, porém números a serem analisados, também informa qual o tipo de análise que deve ser feito com esses números. O programa ainda vai das opções de análise disponível, a opção inválida, avisando o usuário sobre a escolha (utilizando o bloco default).

Outro conjunto de ferramentas importante para a programação são os Laços, também chamados de Loops. Em linguagem C# tem

- • for
- • foreach
- • while
- • do while

O laço for possui três componentes para controle de suas iterações (voltas do laço), sendo:

- • O primeiro componente refere-se a declaração da variável de controle do laço.
- • O segundo componente é a condição de parada do laço. É a expressão lógica, com base na variável de controle do laço execução do laço (cada iteração).
- • O terceiro componente é a manipulação da variável de controle do laço.

Vejamos um exemplo:

```

static void Main(string[] args)
{
    for (int i = 0; i < 5; i++)
    {
        Console.WriteLine("Iteração: {0}", i);
    }
}

```

### Código 12-testafor

No exemplo Testafor, temos um laço for onde a variável de controle do laço (também chamada de variável contadora) foi declarada e dada por  $i < 5$ , ou seja, no momento em que a variável de controle do laço chegar no valor 5, o laço não mais será executado. E a contadora se dará por um auto incremento de uma unidade ( $i++$ ) em relação ao valor anterior.

Deste modo, será apresentada ao usuário uma contagem de zero a quatro na tela (Figura 27).

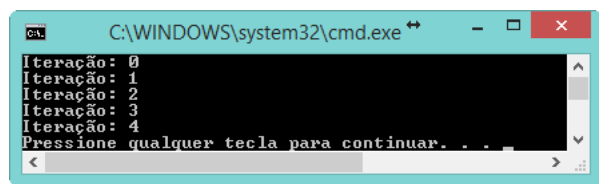
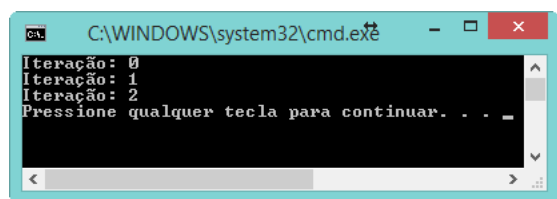


Figura 27-Saída das iterações do laço for.

O laço for permite alguns controles secundários em relação as suas iterações. Vejamos:

```
static void Main(string[] args)
{
    for (int i = 0; i < 5; i++)
    {
        if (i == 3)
        {
            break;
        }
        Console.WriteLine("Iteração: {0}", i);
    }
}
```

Testafor2



Interrompendo as iterações

```
static void Main(string[] args)
{
    for (int i = 0; i < 5; i++)
    {
        if (i == 3)
        {
            continue;
        }
        Console.WriteLine("Iteração: {0}", i);
    }
}
```

Código 14-testfor3

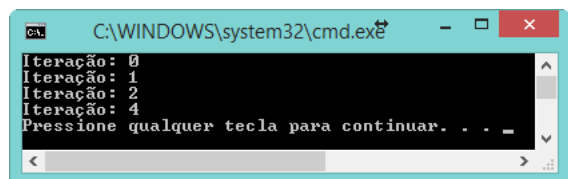


Figura 29-Programa desviando para o fim do laço.

Os programas Testafor2 e Testafor3 apresentam o uso dos comandos break e continue dentro do laço for.

O comando break interrompe o laço e dando continuidade ao programa. No exemplo Testafor2, o laço será interrompido quando a 28).

Já o comando `continue` interrompe a iteração corrente, não o laço todo. No exemplo `Testafor3`, quando a variável contadora chega executada, mas as próximas iterações serão executadas normalmente até o final do laço ( $i < 5$ ) (Figura 29).

Vejamos o código a seguir:

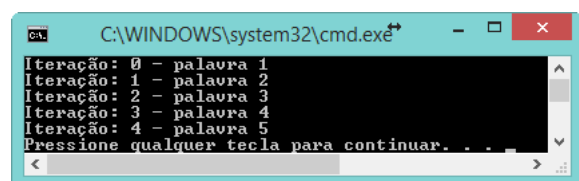
```
static void Main(string[] args)
{
    string[] Texto = new string[5];
    Texto[0] = "palavra 1";
    Texto[1] = "palavra 2";
    Texto[2] = "palavra 3";
    Texto[3] = "palavra 4";
    Texto[4] = "palavra 5";
    int i = 0;
    foreach (string s in Texto)
    {
        Console.WriteLine("Iteração: {0} - {1}", i, s);
        i++;
    }
}
```

testafor3

O programa `Testafor4` traz a aplicação de duas técnicas interessantes: a utilização de vetores (*arrays*) e de um laço específico para

Em C#, os vetores são objetos e como tal, são declarados e instanciados (veremos posteriormente mais detalhes sobre declaração caso, foi declarado um vetor chamado `textos` de cinco posições (`string[] textos = new string[5]`). Em cada posição do vetor (lembrando de zero) foi atribuído um valor do tipo `string` (um texto). Por fim, foi declarada uma variável que será utilizada como variável contadora.

O laço `foreach` indicado em casos onde se faz necessário percorrer todos os elementos de um vetor, pois sua condição de parada é a de percorrer todas as posições do vetor. Para isso o laço `foreach` solicita que seja passada como parâmetro uma variável de controle para capturar o elemento do vetor, tratando assim os dados daquele elemento como uma simples variável (`string s in textos`). Perceba que a variável contadora `i` é utilizada para mostrar em que posição está o laço. Para o usuário será mostrado o seguinte resultado (Figura 30):



Saída das iterações do comando `foreach`.

O próximo laço que estudaremos é o `while`. O laço `while` é usado quando não sabemos o número de vezes que devemos executar o bloco dentro do `while`. Essa condição é uma expressão booleana que deverá ser verdadeira para a primeira ou a próxima iteração (Lima e Reis, 2002).

```
static void Main(string[] args)
{
    int i = 1;
    while (i < 10)
    {
        Console.WriteLine("Iteração: {0}", i);
        i++;
    }
}
```

Testawhile

O exemplo `Testawhile` mostra uma contagem de um a noventa e nove. A condição de parada do laço é  $i < 10$ , logo, a saída apresentada será:

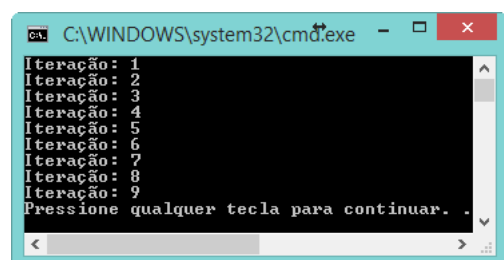


Figura 31-Saída do programa `TestaWhile`

Quando a variável `i` chegar ao valor de cem, a condição do laço será falsa, indicando assim que é o momento de terminar o laço e os comandos do laço `while`.

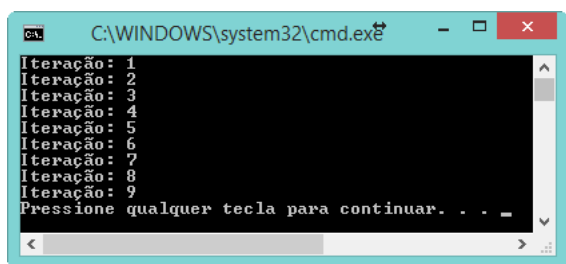
O laço *while* permite o uso dos comandos *break* e *continue*, da mesma forma do laço *for* (já explicado anteriormente).

Por fim, o laço *do-while* é usado quando queremos que um bloco de código seja executado pelo menos uma vez. A condição a ser de código e não no começo, como no caso dos laços *for* e *while* (LIMA & REIS, 2002))

```
static void Main(string[] args)
{
    int i = 1;
    do
    {
        Console.WriteLine("Iteração: {0}", i);
        i++;
    } while (i < 10);
}
```

#### Testadownhile

O programa *Testadownhile* realiza a mesma contagem que o programa *Testawhile*, porém o comportamento apresentado ao usuário primeiro executa o bloco de comandos para depois verificar a condição do laço, a saída será apresentada ao usuário será (Figura 3



Saída do programa *Testadownhile*.

Em resumo, a diferença entre o laço *while* e o laço *do-while* é que o primeiro testa a condição e depois executa o bloco de comandos e depois testa a condição.

Quando queremos guardar diversos objetos, informações, podemos fazer uso de Matrizes. Uma matriz (*array*) é uma estrutura de elementos e ainda nos possibilita pegar esses elementos de maneira fácil!

Criar uma matriz de 5 posições de números inteiros:

```
int[] numeros = new int[5];
```

Para guardar elementos nessas posições, fazemos:

```
numeros[0] = 1;
numeros[1] = 600;
numeros[2] = 257;
numeros[3] = 12;
numeros[4] = 42;
Console.WriteLine(numeros[1]);
```

A primeira posição de um *array* é 0 (zero). Logo, as posições de um *array* vão de 0 (zero) até (tamanho-1).

Podemos criar e logo em seguida inicializar o conteúdo de uma *array*, para isso, o C# nos oferece um atalho para criar e inicializar um *array* de inteiros preenchido com os números de 1 a 5, poderíamos utilizar o seguinte código:

```
int[] umAoCinco = new int[] { 1, 2, 3, 4, 5 };
```

A manipulação de um *array* não é fácil. Por exemplo, em um *array* com 5 elementos valores armazenados. Se quisermos remover todo nosso *array*. Para inserir um elemento no meio do *array* precisamos "abrir um espaço" dentro da *array*, empurrando elementos para o lado. Para colocar o novo elemento no meio, isto se tivermos espaço livre dentro a *array*.

Para contornarmos as dificuldades de trabalhar com uma *array*, o C# oferece uma classe chamada *List*. Não nos preocupemos com enquanto será apenas uma técnica que pode ser utilizada sem o conhecimento de OO. Para utilizarmos uma lista dentro do código elemento que a lista armazenará:

```
List<int> lista = new List<int>();
```

Da mesma forma que criamos a lista de inteiros, também poderíamos criar uma lista de qualquer outro tipo ou classes do C#. Esses elementos dentro de um *array*.

Agora que instanciamos o *List*, podemos utilizar o método *Add* para armazenar novos elementos:

```
// c1 fica na posição 0  
lista.Add(1);  
// c2 na 1  
lista.Add(2);  
// e c3 na 2  
lista.Add(3);
```

Se acessar um elemento fazemos como no *array*, ou seja-la pela sua posição (no caso, 0, igual no *array*):

```
int j = lista[0];
```

Se quisermos remover uma das contas da lista, podemos usar o método *Remove* ou *RemoveAt*:

```
lista.Remove(2); // remove pelo elemento  
lista.RemoveAt(0); // remove pelo índice
```

Se quisermos saber quantos elementos existem em nosso List, podemos simplesmente ler a propriedade Count:

```
int qtdDeElementos = lista.Count;
```

Também podemos descobrir se um elemento está dentro de uma lista:

```
bool tem1 = lista.Contains(1); // true  
bool tem7 = lista.Contains(7); // false
```

Um outro recurso que a classe List nos fornece é a iteração em cada um dos seus elementos pelo *foreach*:

```
foreach (int i in lista)  
{  
    Console.WriteLine(i);  
}
```

---

### Exercício 1:

B1-3 1

Sobre a plataforma .Net, é correto afirmar :

A)

A plataforma .Net foi desenvolvida para uma combinação hardware/software particular.

B)

Em vez de serem desenvolvidos para uma combinação hardware/software particular, os programas serão desenvolvidos para o ".N onde o .NET estiver implementado.

C)

Os componentes que compõem a plataforma .NET são chamados coletivamente de Common Language Runtime(CLR).

D)

Em vez de serem desenvolvidos para uma combinação hardware/software particular, os programas serão desenvolvidos para o ".N dependem da instalação do .Net Framework.

E)

.NET é uma "plataforma de hardware"

---

### Exercício 2:

Considere as afirmações abaixo:

I.A CLR faz a conversão para a linguagem Assembly

II.A CLR gerencia o uso da CPU

III.Uma das funções da CLR é o carregamento e execução de programas, com controle de versão e outras características.

Assinale a alternativa correta

A)

Somente a afirmação I está correta

B)

Somente a afirmação II está correta

C)

Somente a afirmação III está correta

D)

Mais de uma afirmação está correta

E)

nenhuma das afirmações está correta

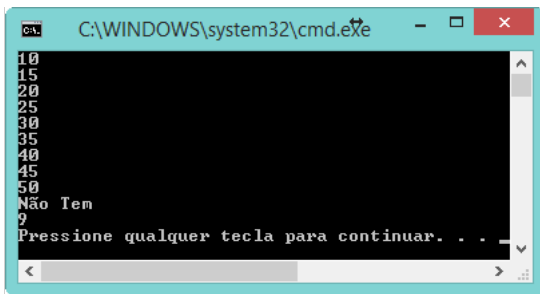
### Exercício 3:

Considere o código abaixo:

```
static void Main(string[] args)
{
    string tem1;
    List<int> lista = new List<int>();
    for (int i = 0; i <= 50; i++)
    {
        if ((i % 5) == 0)
        {
            lista.Add(i);
        }
    }
    lista.Remove(0);
    lista.RemoveAt(0);
    int qtdDeElementos = lista.Count;
    if (lista.Contains(5))
        tem1 = "Tem";
    else
        tem1 = "Não Tem";
    foreach (int i in lista)
    {
        Console.WriteLine(i);
    }
    Console.WriteLine("{1}\n{0}", qtdDeElementos, tem1);
}
```

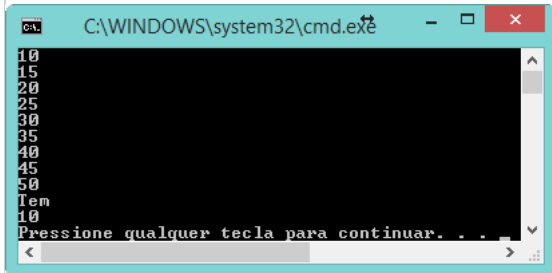
Assinale a alternativa que contém a saída correta do programa.

A)



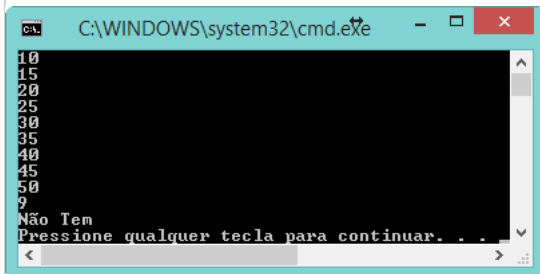
```
C:\WINDOWS\system32\cmd.exe
10
15
20
25
30
35
40
45
50
Não Tem
Pressione qualquer tecla para continuar. . .
```

B)



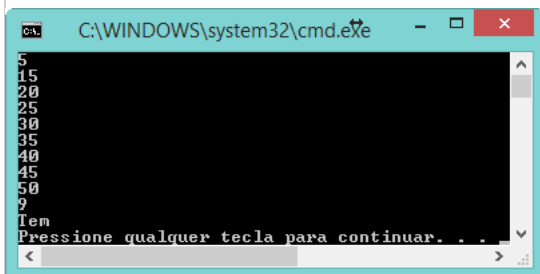
```
C:\WINDOWS\system32\cmd.exe
10
15
20
25
30
35
40
45
50
Tem
Pressione qualquer tecla para continuar. . .
```

C)



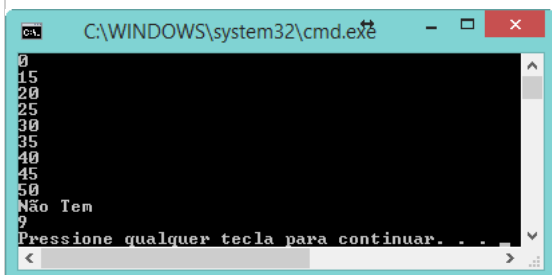
```
C:\WINDOWS\system32\cmd.exe
10
15
20
25
30
35
40
45
50
Não Tem
Pressione qualquer tecla para continuar. . .
```

D)



```
C:\WINDOWS\system32\cmd.exe
5
15
20
25
30
35
40
45
50
Tem
Pressione qualquer tecla para continuar. . .
```

E)



```
C:\WINDOWS\system32\cmd.exe
0
15
20
25
30
35
40
45
50
Não Tem
Pressione qualquer tecla para continuar. . .
```

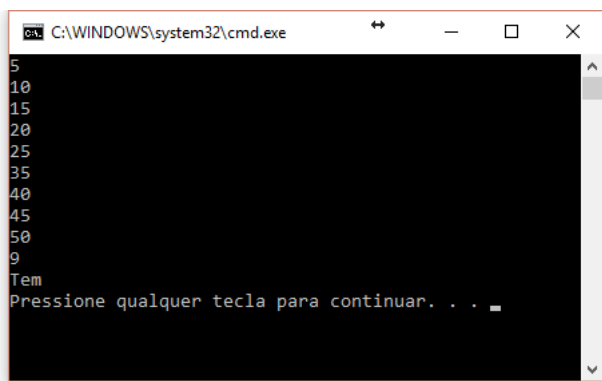
**Exercício 4:**

Considere o programa abaixo:

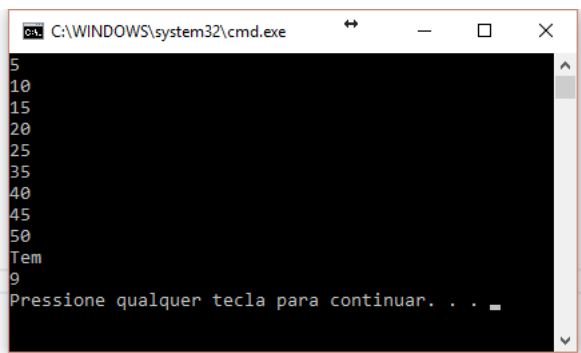
```
static void Main(string[] args)
{
    string tem1;
    List<int> lista = new List<int>();
    for (int i = 0; i <= 50; i++)
    {
        if ((i % 5) == 0)
        {
            lista.Add(i);
        }
    }
    lista.RemoveAt(5);
    int qtdDeElementos = lista.Count;
    if (lista.Contains(5))
        tem1 = "Tem";
    else
        tem1 = "Não Tem";
    foreach (int i in lista)
    {
        Console.WriteLine(i);
    }
    Console.WriteLine("{1}\n{0}", qtdDeElementos, tem1);
}
```

Assinale a alternativa que contém a saída correta do programa.

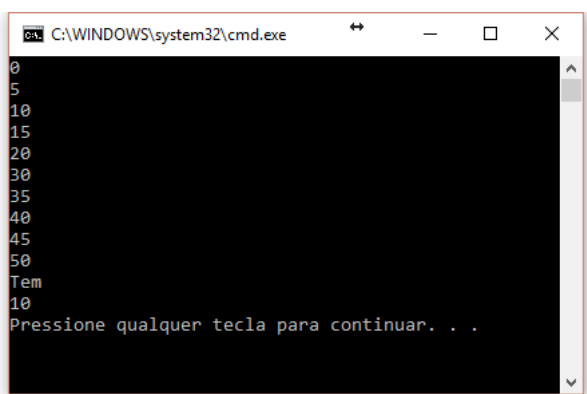
A)



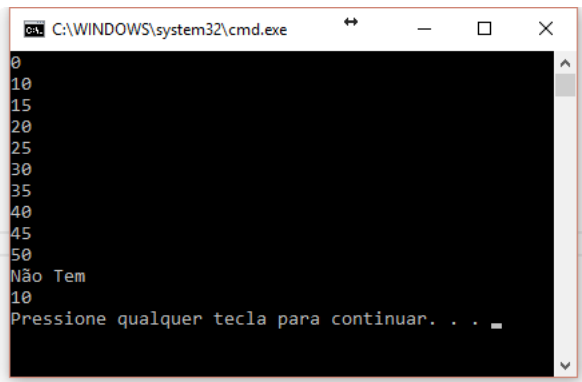
B)



C)



D)

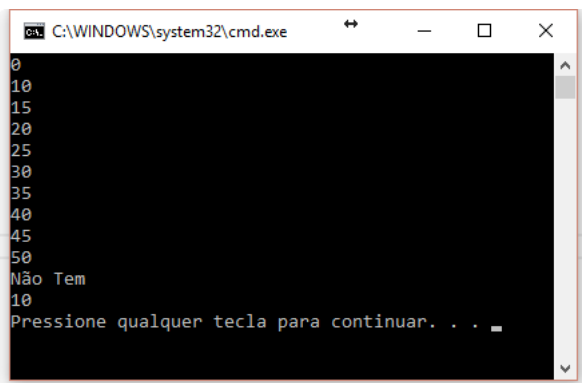


```

C:\WINDOWS\system32\cmd.exe
0
10
15
20
25
30
35
40
45
50
Não Tem
10
Pressione qualquer tecla para continuar. . .

```

E)



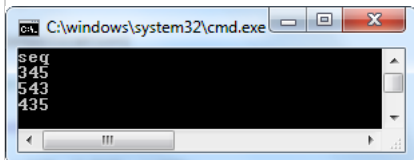
```

C:\WINDOWS\system32\cmd.exe
0
10
15
20
25
30
35
40
45
50
Não Tem
10
Pressione qualquer tecla para continuar. . .

```

**Exercício 5:**

Dada a seguinte tela de saída



```

C:\windows\system32\cmd.exe
seq
345
543
435

```

Qual dos programas abaixo NÃO mostrou esta saída?

A)

```

static void Main(string[] args)
{
    int a=345, b=543, c=435;
    Console.WriteLine("seq\n{1}\n{0}\n{2}", b,a,c);
    Console.ReadKey();
}

```

B)

```

static void Main(string[] args)
{
    int a=345, b=543, c=435;
    Console.WriteLine("seq\n{0}\n{1}\n{2}", a,b,c);
}

```

```
    Console.ReadKey();  
}
```

C)

```
static void Main(string[] args)  
{  
    int a=345, b=435, c=543;  
    Console.WriteLine("seq\n{0}\n{2}\n{1}", a,b,c);  
    Console.ReadKey();  
}
```

D)

```
static void Main(string[] args)  
{  
    int a=543, b=345, c=435;  
    Console.WriteLine("seq\n{1}\n{0}\n{2}", a,b,c);  
    Console.ReadKey();  
}
```

E)

Todas Corretas ou Mais de uma incorreta

**Exercício 6:**

Considere o programa abaixo:

```
static void Main(string[] args)  
{  
    int a;  
    a = Console.ReadLine();  
    a = a % 19;  
    if (a <= 19)  
    {  
        Console.WriteLine("é menor");  
    }  
    else  
        Console.WriteLine("é maior");  
}
```

Assinale a alternativa com a afirmação correta:

A)

O programa não compila pois não existe o operador %.

B)

O programa não compila pois está com os blocos desbalanceados

C)

O programa não compila pois é necessária uma conversão de tipo na leitura

D)

O programa compila normalmente mas o resultado é sempre "é menor"

E)

O programa compila normalmente mas o resultado é sempre "é maior"

### Exercício 7:

B1-4 5

Dentre os comandos do C# qual **não** é utilizado para a programação de laços?

A)

switch case

B)

for

C)

while

D)

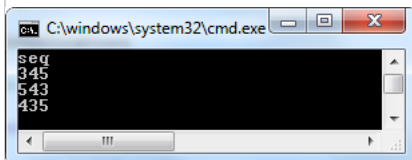
foreach

E)

do while

### Exercício 8:

Considerando a tela abaixo



tela 1

I.

```
static void Main(string[]  
args)  
{  
    int a=345, b=543, c=435;  
    Console.WriteLine("seq\n{1}\n{0}\n{2}", b,a,c);  
    Console.ReadKey();  
}
```

II.

```
static void Main(string[]  
args)  
{  
    int a=345, b=543, c=435;  
    Console.WriteLine("seq\n{0}\n{1}\n{2}", a,b,c);  
}
```

```
Console.ReadKey();  
}  
  
III.  
  
static void Main(string[]  
args)  
{  
    int a=345, b=435, c=543;  
    Console.WriteLine("seq\n{0}\n{2}\n{1}", a,b,c);  
    Console.ReadKey();  
}
```

- A)  
Somente o programa I gera a tela 1
- B)  
Somente o programa II gera a tela 1
- C)  
Somente o programa III gera a tela 1
- D)  
Mais de um programa geram a tela 1
- E)  
Nenhum dos programas geram o tela 1